TOPIC 1 : INTRODUCTION

11. Given an m x n grid and a ball at starting cell (i, j), find the number of ways to move the ball out of grid boundary in exactly N steps. Example 1: Input: m = 2, n = 2, N = 2, i = 0, j = 0 -> Output: 6 Example 2: Input: m = 1, n = 3, N = 3, i = 0, j = 1 -> Output: 12

Aim

To write a Python program using Dynamic Programming / Memoization to count out-of-boundary paths in grid.

Algorithm

1. Start.
2. Define DFS function dfs(r, c, steps).
3. If r < 0 or r >= m or c < 0 or c >= n, return 1 (ball moved out).
4. If steps == 0, return 0 (no moves left).
5. Sum paths in 4 directions: up, down, left, right.
6. Use @lru_cache for memoization.
7. Return total paths modulo 10■+7.
8. Stop.

main.py Run

```python
from functools import lru_cache

def findPaths(m, n, N, i, j):
    MOD = 10**9 + 7

    @lru_cache(None)
    def dfs(r, c, steps):
        if r < 0 or r >= m or c < 0 or c >= n:
            return 1
        if steps == 0:
            return 0
        return (dfs(r + 1, c, steps - 1) +
                dfs(r - 1, c, steps - 1) +
                dfs(r, c + 1, steps - 1) +
                dfs(r, c - 1, steps - 1)) % MOD

    return dfs(i, j, N)

print(findPaths(2, 2, 2, 0, 0))
print(findPaths(1, 3, 3, 0, 1))
```

Output

```
6
12
```

Time Complexity

O(m × n × N) states in DP table.

Space Complexity

O(m × n × N) space for memoization table.

12. You are a professional robber planning to rob houses arranged in a circle. Adjacent houses have connected security systems. Calculate maximum money you can rob without alerting police. Example 1: Input: nums = [2, 3, 2] -> Output: 3 Example 2: Input: nums = [1, 2, 3, 1] -> Output: 4

Aim

To write a Python program to solve House Robber II dynamic programming problem for circular array.

Algorithm

1. Start.
2. If len(nums) == 1, return nums[0].
3. Define helper function rob_linear(arr) for non-circular array.
4. Compare max(rob_linear(nums[:-1]), rob_linear(nums[1:])).
5. Return maximum robbed amount.
6. Stop.

main.py Run

```python
def robLinear(nums):
    prev1, prev2 = 0, 0
    for money in nums:
        temp = max(prev1, prev2 + money)
        prev2 = prev1
        prev1 = temp
    return prev1

def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]
    return max(robLinear(nums[:-1]), robLinear(nums[1:]))

print(rob([2, 3, 2]))
print(rob([1, 2, 3, 1]))
```

Output

```
3
4
```

Time Complexity

O(n) linear scan over two passes.

Space Complexity

O(1) constant auxiliary space.

13. You are climbing a staircase. It takes n steps to reach top. Each time you can climb 1 or 2 steps. In how many distinct ways can you climb to the top? Example 1: Input: n = 4 -> Output: 5 Example 2: Input: n = 3 -> Output: 3

Aim

To write a Python program using Dynamic Programming / Fibonacci pattern to find number of ways to climb n stairs.

Algorithm

1. Start.
2. If n <= 2, return n.
3. Maintain first = 1 and second = 2.
4. Loop from step 3 to n: compute third = first + second, then update first = second, second = third.
5. Return second.
6. Stop.

main.py Run

```python
def climbStairs(n):
    if n <= 2:
        return n
    first, second = 1, 2
    for i in range(3, n + 1):
        third = first + second
        first = second
        second = third
    return second

print(climbStairs(4))
print(climbStairs(3))
```

Output

```
5
3
```

Time Complexity

O(n) time complexity.

Space Complexity

O(1) constant auxiliary space.

14. A robot is located at top-left corner of m x n grid. Robot can only move down or right. How many possible unique paths are there to reach bottom-right corner? Example 1: Input: m = 7, n = 3 -> Output: 28 Example 2: Input: m = 3, n = 2 -> Output: 3

Aim

To write a Python program using 1D DP table to calculate unique grid paths.

Algorithm

1. Start.
2. Initialize 1D DP array dp of size n filled with 1s.
3. Loop for row from 1 to m-1.
4. Loop for col from 1 to n-1: dp[col] += dp[col-1].
5. Return dp[-1].
6. Stop.

main.py Run

```
def uniquePaths(m, n):
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[-1]

print(uniquePaths(7, 3))
print(uniquePaths(3, 2))
```

Output

```
28
3
```

Time Complexity

O(m × n) time complexity.

Space Complexity

O(n) space complexity.

15. In a string S of lowercase letters, groups of consecutive identical characters of length >= 3 are large groups. Return start and end indices of all large groups sorted by start index. Example 1: Input: s = "abbxxxxzzy" -> Output: [[3,6]] Example 2: Input: s = "abc" -> Output: []

Aim

To write a Python program to locate intervals of large character groups in a string.

Algorithm

1. Start.
2. Maintain start pointer = 0.
3. Traverse string with index i from 1 to len(s).
4. If i == len(s) or s[i] != s[start]:
5. If group length (i - start) >= 3, append [start, i - 1] to result.
6. Update start = i.
7. Return result.
8. Stop.

main.py Run

```python
def largeGroupPositions(s):
    result = []
    start = 0
    for i in range(1, len(s) + 1):
        if i == len(s) or s[i] != s[start]:
            if i - start >= 3:
                result.append([start, i - 1])
            start = i
    return result

print(largeGroupPositions("abbxxxxzzy"))
print(largeGroupPositions("abc"))
```

Output

```
[[3, 6]]
[]
```

Time Complexity

O(n) linear scan through string.

Space Complexity

O(1) auxiliary space excluding output list.

16. Given current state of m x n grid board, compute next state according to Conway's Game of Life rules (live cell with <2 or >3 neighbors dies; dead cell with 3 neighbors lives). Example 1: Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] -> Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Aim

To write a Python program to simulate Conway's Game of Life grid transformation.

Algorithm

1. Start.
2. Copy initial board state.
3. For each cell (r, c), count 8-directional live neighbors.
4. Apply 4 rules to update cell state in target board.
5. Return modified board.
6. Stop.

main.py Run

```python
def gameOfLife(board):
    rows = len(board)
    cols = len(board[0])
    temp = [row[:] for row in board]
    directions = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]

    for r in range(rows):
        for c in range(cols):
            live = 0
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and temp[nr][nc] == 1:
                    live += 1
            if temp[r][c] == 1:
                if live < 2 or live > 3:
                    board[r][c] = 0
            else:
                if live == 3:
                    board[r][c] = 1
    return board

board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print(gameOfLife(board))
```

Output

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

Time Complexity

O(m × n) grid traversal.

Space Complexity

O(m × n) auxiliary grid memory.

17. Glasses stacked in pyramid. Each glass holds 1 cup. Excess champagne overflows equally to 2 glasses below. Given poured cups, query_row, and query_glass, return how full target glass is. Example 1: poured = 1, query_row = 1, query_glass = 1 -> Output: 0.00000 Example 2: poured = 2, query_row = 1, query_glass = 1 -> Output: 0.50000

Aim

To write a Python program to compute liquid quantity in champagne tower glass using DP simulation.

Algorithm

1. Start.
2. Initialize 2D array tower of size 101x101 with 0.0.
3. Set tower[0][0] = poured.
4. For row from 0 to query_row:
5. For glass from 0 to row:
6. excess = (tower[row][glass] - 1.0) / 2.0
7. If excess > 0: add excess to tower[row+1][glass] and tower[row+1][glass+1].
8. Return min(1.0, tower[query_row][query_glass]).
9. Stop.

main.py Run

```python
def champagneTower(poured, query_row, query_glass):
    tower = [[0.0] * 101 for _ in range(101)]
    tower[0][0] = poured
    for r in range(query_row + 1):
        for c in range(r + 1):
            if tower[r][c] > 1.0:
                excess = (tower[r][c] - 1.0) / 2.0
                tower[r + 1][c] += excess
                tower[r + 1][c + 1] += excess
    return min(1.0, tower[query_row][query_glass])

print(f"{champagneTower(1, 1, 1):.5f}")
print(f"{champagneTower(2, 1, 1):.5f}")
```

Output

```
0.00000
0.50000
```

Time Complexity

O(query_row²) operations.

Space Complexity

O(query_row²) grid memory.

TOPIC 2 : BRUTE FORCE

18. Write a program to perform list processing on test cases: - An empty list: [] -> Expected Output: [] - A list with one element: [1] -> Expected Output: [1] - A list with all identical elements: [7, 7, 7, 7] -> Expected Output: [7, 7, 7, 7] - A list with negative numbers: [-5, -1, -3, -2, -4] -> Expected Output: [-5, -4, -3, -2, -1]

Aim

To write a Python program using brute force techniques to safely process list operations under various edge cases.

Algorithm

1. Start.
2. Read input list.
3. Check if list is empty or has single element. If so, return list as is.
4. Sort list using brute force comparison passes.
5. Return processed list.
6. Stop.

main.py Run

```python
def process_list(arr):
    if not arr or len(arr) == 1:
        return arr
    n = len(arr)
    res = list(arr)
    for i in range(n):
        for j in range(i + 1, n):
            if res[i] > res[j]:
                res[i], res[j] = res[j], res[i]
    return res

print("Empty:", process_list([]))
print("One Element:", process_list([1]))
print("Identical:", process_list([7, 7, 7, 7]))
print("Negative:", process_list([-5, -1, -3, -2, -4]))
```

Output

```
Empty: []
One Element: [1]
Identical: [7, 7, 7, 7]
Negative: [-5, -4, -3, -2, -1]
```

Time Complexity

$O(n^2)$ brute force sorting comparisons.

Space Complexity

O(n) auxiliary list storage.

19. Describe Selection Sort's process of sorting an array. Implement Selection Sort and test on: 1. Random Array: [5, 2, 9, 1, 5, 6] -> Output: [1, 2, 5, 5, 6, 9] 2. Reverse Sorted Array: [10, 8, 6, 4, 2] -> Output: [2, 4, 6, 8, 10] 3. Already Sorted Array: [1, 2, 3, 4, 5] -> Output: [1, 2, 3, 4, 5]

Aim

To write a Python program to implement Selection Sort algorithm and trace its minimum element selection process.

Algorithm

1. Start.
2. Iterate i from 0 to n-1.
3. Assume min_idx = i.
4. Inner loop j from i+1 to n-1: if arr[j] < arr[min_idx], set min_idx = j.
5. Swap arr[i] with arr[min_idx].
6. Return sorted array.
7. Stop.

main.py Run

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

print("Random:", selection_sort([5, 2, 9, 1, 5, 6]))
print("Reverse:", selection_sort([10, 8, 6, 4, 2]))
print("Sorted:", selection_sort([1, 2, 3, 4, 5]))
```

Output

```
Random: [1, 2, 5, 5, 6, 9]
Reverse: [2, 4, 6, 8, 10]
Sorted: [1, 2, 3, 4, 5]
```

Time Complexity

$O(n^2)$ time complexity in all cases (Best, Average, Worst).

Space Complexity

O(1) in-place space complexity.

20. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed. Test Cases: 1. [64, 25, 12, 22, 11] -> Expected: [11, 12, 22, 25, 64] 2. [1, 2, 3, 4, 5] (Already sorted) -> Expected: [1, 2, 3, 4, 5] 3. [5, 4, 3, 2, 1] (Reverse sorted) -> Expected: [1, 2, 3, 4, 5]

Aim

To write a Python program for Bubble Sort with an early-exit flag when array becomes sorted.

Algorithm

1. Start.
2. Outer loop i from 0 to n-1.
3. Set swapped = False.
4. Inner loop j from 0 to n-i-2: if arr[j] > arr[j+1], swap and set swapped = True.
5. If swapped is False at end of inner pass, break out of loop.
6. Return sorted list.
7. Stop.

main.py Run

```python
def optimized_bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

print("Test 1:", optimized_bubble_sort([64, 25, 12, 22, 11]))
print("Already sorted:", optimized_bubble_sort([1, 2, 3, 4, 5]))
print("Reverse sorted:", optimized_bubble_sort([5, 4, 3, 2, 1]))
```

Output

```
Test 1: [11, 12, 22, 25, 64]
Already sorted: [1, 2, 3, 4, 5]
Reverse sorted: [1, 2, 3, 4, 5]
```

Time Complexity

Best Case: O(n), Worst Case: O(n^2).

Space Complexity

O(1) auxiliary space.